

Calcolatori Elettronici

18 giugno 2025

Teoria

Esercizio 1 (5 punti): Si realizzi una rete sequenziale in grado di determinare se, in una sequenza infinita di caratteri appartenenti all'alfabeto $I = \{a, b\}$, un gruppo di tre caratteri consecutivi è costituito da tutti e soli caratteri identici. Per ciascun gruppo di 3 caratteri, la rete restituirà come output il valore `si` o `no`. Fino a che non è stato ricevuto un gruppo di 3 caratteri, l'uscita assumerà il valore `?`. Si realizzi il circuito equivalente dell'equazione di eccitazione minimizzata di un flip flop di stato a scelta.

Esercizio 2 (10 punti): Si consideri il seguente frammento di codice in assembly x86 eseguito da un processore z64 con pipeline. Illustrare di quali criticità soffre questo frammento di codice e inserire gli stalli necessari a risolverle.

```
1  movl (%rdi), %eax
2  leal (%rsi,%rax), %r8d
3  movl %r8d, 4(%rdi)
4  subl %edx, %eax
```

Esercizio 3 (10 punti): Convertire in C il seguente frammento di codice assembly.

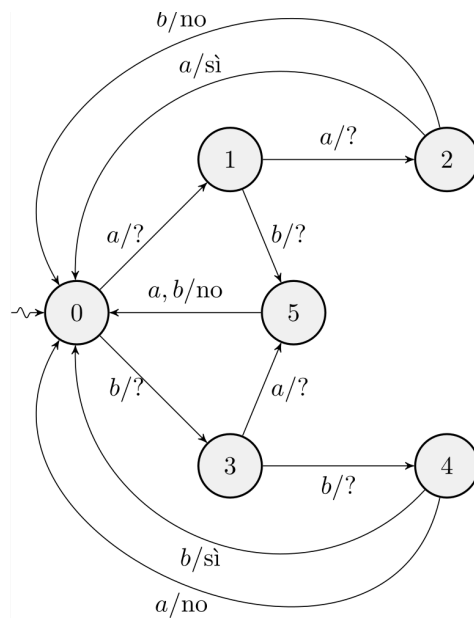
```
1  whatsthis:
2      mov $1, %rax
3      cmp $1, %rdi
4      jc .done
5      jz .done
6  .loop:
7      imulq %rdi, %rax
8      subq $1, %rdi
9      cmpq $1, %rdi
10     jnc .loop
11     jz .loop
12  .done:
13     ret
```

Esercizio 4 (5 punti): Illustrare vantaggi e svantaggi della cache set associativa ad n vie confrontandola con la cache completamente associativa.

Soluzioni

Esercizio 1

L'automa deve contare il numero di caratteri uguali. Poiché i gruppi sono da tre, se il secondo è diverso, occorre aspettare un altro carattere, qualsiasi esso sia: l'uscita finale sarà comunque `no`. Si ottiene quindi il seguente automa:



Assumendo di codificare gli stati con il corrispondente valore binario, di codificare $a=0$, $b=1$ per gli ingressi e $no=00$, $sì=01$, $?=10$ per le uscite, la tabella degli stati e transizioni è la seguente.

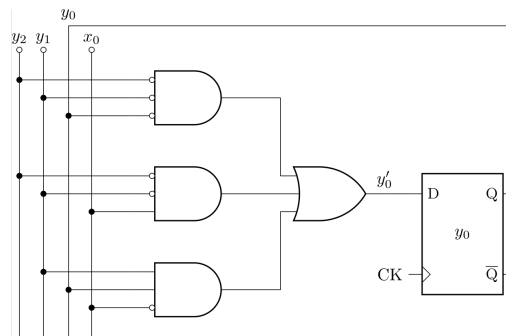
y_2	y_1	y_0	x_0	y'_2	y'_1	y'_0	z_1	z_0
0	0	0	0	0	0	1	1	0
0	0	0	1	0	1	1	1	0
0	0	1	0	0	1	0	1	0
0	0	1	1	1	0	1	1	0
0	1	0	0	0	0	0	0	1
0	1	0	1	0	0	0	0	0
0	1	1	0	1	0	1	1	0
0	1	1	1	1	0	0	1	0
1	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	1
1	0	1	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0

Le rimanenti righe sono associate a d.c.c..

Volendo studiare l'equazione di eccitazione del flip flop associato a y_0 , otteniamo la seguente mappa di Karnaugh:

y_2y_1 y_0x	00	01	11	10
00	1	0	-	0
01	1	0	-	0
11	1	0	-	0
10	0	1	-	0

che determina quindi l'equazione di eccitazione $y'_0 = \overline{y_2y_1y_0} + \overline{y_2y_1}x + y_1y_0\overline{x}$, corrispondente al circuito equivalente:



Esercizio 2

La prima istruzione effettua un accesso in lettura alla memoria e scrive il dato letto in `%eax`. Poiché il valore alimenta l'operando sorgente della successiva `lea`, si manifesta una criticità *read-after-write* di tipo *load-use*: la pipeline non possiede ancora il dato quando l'istruzione aritmetica entra nello stadio EX, dunque l'unico rimedio è introdurre uno stallo fra i due comandi.

La seconda criticità nasce fra il valore calcolato da `lea` e il registro sorgente che la successiva `mov` impiega per scrivere in memoria. In questo caso il risultato dell'ALU è pronto già al termine dello stadio EX, quindi un percorso di forwarding EX→MEM consente alla `mov` di ricevere il valore corretto. In alternativa, se non si assume la presenza di forwarding, è necessario inserire un secondo stallo.

L'ultima dipendenza possibile riguarda ancora il dato inizialmente caricato dalla memoria nella prima istruzione ed utilizzato dalla `subl`. Assumendo che siano state risolte le criticità strutturali sul register file, il valore corretto viene visto dalla `subl`.

Quindi, l'unico stallo essenziale è tra la prima e la seconda istruzione. Esso può essere realizzato con una `nop` a software, o inserendo una bolla in hardware.

Esercizio 3

Viene utilizzato il registro `%rax`, quindi il valore di ritorno sarà un `unsigned long long`. Allo stesso modo, viene utilizzato il registro `%rdi`, pertanto la funzione accetta un unico parametro `unsigned long long`. Il ciclo termina quando il valore contenuto in `%rdi` arriva a 1, ma il codice non entra in `.loop` se `%rdi` non è almeno 1. Si tratta quindi di un ciclo `while`, che implementa il controllo di condizione $(\%rdi > 1)$ prima di entrare nel corpo del ciclo.

All'interno del ciclo, il programma effettua la moltiplicazione tra `%rax` (precedentemente inizializzato a 1) e `%rdi`, che viene poi decrementato di 1. Si tratta quindi di un programma che calcola, come segue, l' n -esimo fattoriale:

```

1 unsigned long long factorial(unsigned long long n)
2 {
3     unsigned long long result = 1;
4     while (n > 1) {
5         result *= n;
6         --n;
7     }
8     return result;
9 }

```

Progetto

Un processore `z64` controlla una linea automatica per lo smistamento di pacchi, composta dalle seguenti periferiche. Il nastro trasportatore (`NASTRO`) è una periferica sincrona il cui stato di avanzamento può essere attivato o interrotto mediante opportune operazioni sulla sua interfaccia. `FOTOCELLULA`, invece, opera in modo asincrono: ogni volta che un pacco attraversa il suo fascio ottico, genera una richiesta di interruzione verso il processore. Contestualmente, è possibile leggere da un registro di stato il codice di spedizione del pacco rilevato, rappresentato come longword. L'attuatore pneumatico `SPINTA`, anch'esso a funzionamento sincrono, consente di deviare il pacco verso un binario laterale quando viene attivato.

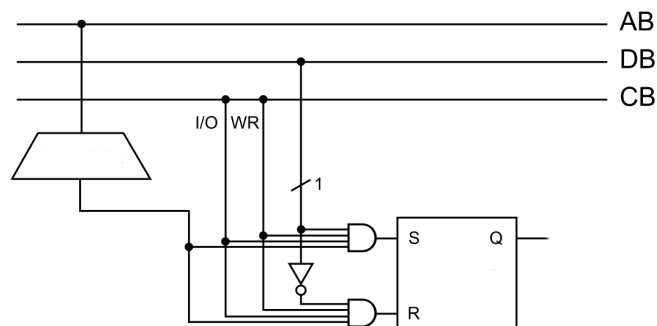
In memoria principale è presente una tabella denominata `destinazioni`, organizzata come vettore di 256 word. Quando lo `z64` acquisisce da `FOTOCELLULA` un codice di spedizione, estrae da esso la word più significativa e la utilizza come indice per accedere all'elemento corrispondente nella tabella. Il contenuto della cella può essere pari a 0 oppure a 1: il valore 0 indica che il pacco deve proseguire sul binario principale, mentre il valore 1 prescrive la deviazione sul binario laterale. In quest'ultimo caso, lo `z64` arresta `NASTRO`, attiva `SPINTA` e attende il completamento della deviazione prima di riavviare l'avanzamento del nastro.

Realizzare:

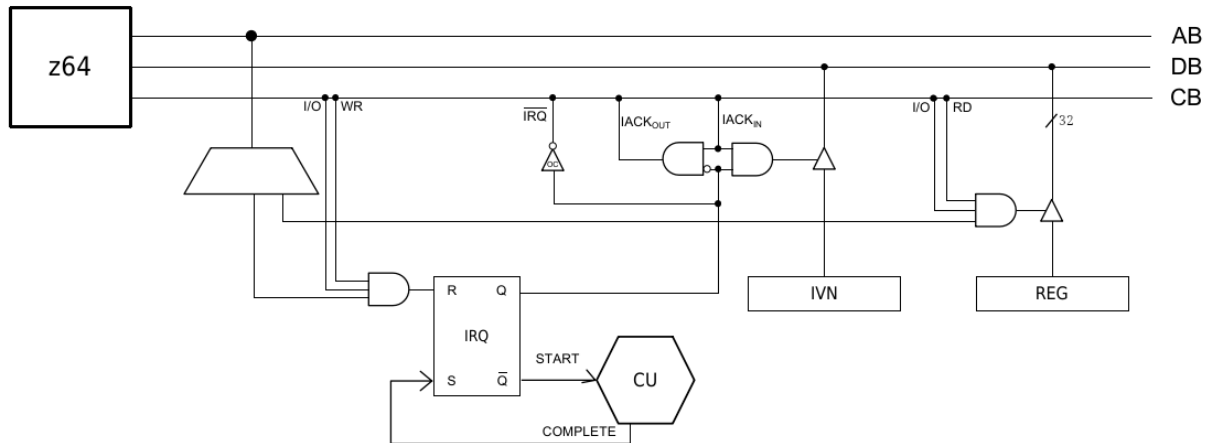
- Le interfacce dei dispositivi `NASTRO`, `FOTOCELLULA` e `SPINTA`;
- Tutto il codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

Soluzione

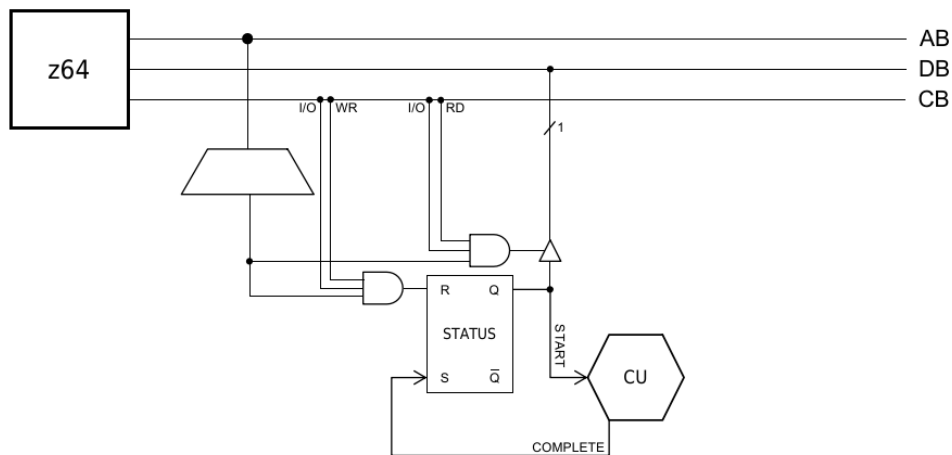
La periferica `NASTRO` è sincrona e deve permettere l'attivazione/disattivazione del movimento del nastro. Può quindi essere realizzata come un semplice "interruttore":



La periferica **FOTOCELLULA** è asincrona, pertanto deve essere in grado di inviare richieste di interruzione. Inoltre, deve essere in grado di permettere la lettura del codice del pacco, necessitando quindi di un registro di interfaccia apposito. La sua interfaccia può quindi essere:



La periferica **SPINTA** è una periferica sincrona. Occorre attendere che il pacco, una volta attivato il dispositivo, sia stato spostato completamente. Pertanto, è necessario attendere a software il completamento dell'operazione richiesta. L'interfaccia può quindi essere:



Il codice per far funzionare il sistema può essere il seguente.

```

1  .org 0x800
2  .data
3      .equ FOTOCELLULA_REG, 0x10
4      .equ FOTOCELLULA_IRQ, 0x20
5      .equ NASTRO, 0x30
6      .equ SPINTA, 0x40
7      destinazioni: .fill 256, 2
8
9  .text
10  main:
11      # avvio iniziale del nastro
12      movb $1, %al
13      outb %al, $NASTRO
14      sti

```

```

15     .loop:
16         hlt
17         jmp loop
18
19
20     .driver 0 # Fotocellula
21         push %rax
22         push %rcx
23         push %rbx
24         inl $FOTOCELLULA_REG, %eax
25
26         # la word più significativa decide la destinazione; ogni entry della tabella
27         # occupa 2 byte, perciò basta mascherare a 8 bit e traslare di 2 byte
28         movl %eax, %ebx
29         shrl $16, %ebx # %ebx ← word alta
30         andl $0xFF, %rbx # %ebx ∈ [0,255]
31
32         # Verifica dove inoltrare il pacco
33         movw destinazioni(,%rbx,2), %cx
34         testw %cx, %cx
35         jz .non_spingere
36
37         # Ferma NASTRO e avvia SPINTA
38         movb $0, %al
39         outb %al, $NASTRO
40         outb %al, $SPINTA
41
42         # Attendi lo spostamento del pacco
43     .bw:
44         inb $SPINTA, %al
45         btb $0, %al
46         jnc .bw
47
48         # Riavvia nastro
49         outb %al, $NASTRO # in uscita dal BW al = 1
50
51     .non_spingere:
52         outb %al, $FOTOCELLULA_IRQ
53         pop %rbx
54         pop %rcx
55         pop %rax
56         iret

```